

Make-Up Assignment

December 4, 2024

Instructions

In this make-up assignment, you are required to implement three methods within a class named `hw_{student_number}`. You are provided with the method names, parameter types, and expected return types. Your task is to write each method according to the given descriptions and ensure they produce the correct output based on the sample inputs provided. Parameter names are flexible and can be chosen by you.

Question 1: Method Name - `reverseArray`

Parameters

- `int[]` (an array of integers)

Returns

- `int[]` array containing the elements of the input array in reverse order.

Description

The `reverseArray` method takes an array of integers and reverses the order of its elements. The method returns a new array with the elements in reversed order. The original array should **not** be modified; instead, create a new array to store the reversed elements.

Examples

- **Input:** `[1, 2, 3, 4, 5]`
- **Output:** `[5, 4, 3, 2, 1]`
- **Input:** `[10, 20, 30]`
- **Output:** `[30, 20, 10]`
- **Input:** `[7]`
- **Output:** `[7]`
- **Input:** `[]`
- **Output:** `[]`

Question 2: Method Name - findPairs

Parameters

- `int []` (an array of integers)
- `int` (a target sum)

Returns

- `int [] []` two-dimensional array containing all unique pairs of numbers that add up to the target sum. Each pair is represented as an array of two integers `[a, b]`, where $a \leq b$. The pairs should not be repeated in different orders.

Description

The `findPairs` method searches through the input array to find all unique pairs of integers that sum up to a specified target value. The method returns a two-dimensional array containing these pairs. Each pair should be sorted in ascending order, and the overall array should **not** contain duplicate pairs.

Examples

- **Input:**
 - Array: `[1, 2, 3, 4, 5]`
 - Target Sum: 5
- **Output:** `[[1, 4], [2, 3]]`
- **Input:**
 - Array: `[2, 2, 3, 3, 4, 4]`
 - Target Sum: 6
- **Output:** `[[2, 4], [3, 3]]`
- **Input:**
 - Array: `[-1, 0, 1, 2, -2, 3]`
 - Target Sum: 1
- **Output:** `[[ -2, 3], [-1, 2], [0, 1]]`
- **Input:**
 - Array: `[5, 5, 5]`
 - Target Sum: 10
- **Output:** `[[5, 5]]`

Question 3: Method Name - isValidPassword

Parameters

- `String` (a password string)

Returns

- `boolean` indicating whether the password meets the specified criteria.

Description

The `isValidPassword` method checks if a given password string meets the following criteria:

1. The password must be at least **10 characters** long.
2. It must contain at least **3 uppercase letters**.
3. It must contain at least **2 digits**.
4. It must contain at least **3 special characters** (characters that are neither letters nor digits).

You are **required** to implement the following **helper methods** to assist in determining whether a character meets certain conditions:

Helper Method 1: `isNumeric`

- **Parameters:**
 - `char` (a single character)
- **Returns:** `boolean` indicating whether the character is a digit between '0' and '9'.
- **Description:** Checks if the given character is a numeric digit.

Helper Method 2: `isUpperCase`

- **Parameters:**
 - `char` (a single character)
- **Returns:** `boolean` indicating whether the character is an uppercase letter.
- **Description:** Checks if the given character is an uppercase alphabetic letter.

Helper Method 3: `isSpecialCharacter`

- **Parameters:**
 - `char` (a single character)
- **Returns:** `boolean` indicating whether the character is a special character (neither a letter nor a digit).
- **Description:** Checks if the given character is a special character.

Examples

- **Input:** "AbcDefGhi!@123"
 - **Output:** true
 - **Explanation:**
 - * Length ≥ 10
 - * Uppercase letters = 3 (A, D, G)
 - * Digits = 3 (1, 2, 3)
 - * Special characters = 3 (!, @, #)
- **Input:** "password123"
 - **Output:** false
 - **Explanation:**
 - * Does not meet the uppercase letters and special characters criteria.
- **Input:** "A1!B2@C3"
 - **Output:** false
 - **Explanation:**
 - * Length = 9, which is less than the required 10 characters.
- **Input:** "Short1!"
 - **Output:** false
 - **Explanation:**
 - * Length $\nless 10$
 - * Insufficient uppercase letters, digits, and special characters.

Instructions

- **Create a class named** `hw_{student_number}` **and implement all the methods as described.**
- **Use the exact method names provided,** including the helper methods in Question 3.
- **Define the parameters with the specified types;** parameter names can be chosen by you.
- **Ensure that each method returns the specified type** as described in the "Returns" section.
- The logic within each method should fulfill the requirements outlined in the descriptions.
- **Implement the helper methods** listed in Question 3 and use them within your `isValidPassword` method.

- **Do not modify the method signatures** other than defining parameter names and return types.
- **Do not use any built-in Java methods** for checking character types beyond what is specified in the helper methods. Instead, utilize the helper methods you implement.
- **Do not include any additional code** outside of the `hw_{student_number}` class and the specified methods.
- **Test your methods** using the provided examples to ensure they work correctly.
- Pay attention to **edge cases**, such as empty arrays, single-element arrays, or passwords that barely meet the criteria.

Submission Requirements

- **Submit a single Java file named** `hw_{student_number}.java` containing your class and the implemented methods.
 - Your code should be **well-formatted** and include **comments** where necessary to explain your logic.
 - **Include any additional test cases** you used to verify your methods' functionality within a `main` method or separate test methods inside the `hw_{student_number}` class.
 - **Ensure your code compiles and runs without errors.**
 - **Handle any potential exceptions or errors gracefully**, ensuring that your methods behave as expected even with unexpected inputs.
-